

Secure Code Review Report

Scaler Support Desk — Application Security Course Assignment

Student Information

Student Name	Debashis Maharana
Roll Number	23BCS10209
Batch / Section	2027
Email ID	debashis.23bcs10209@sst.scaler.com
Static Analysis Tool Used	Semgrep v1.168.0 (community rules: p/python, p/flask, p/security-audit, p/owasp-top-ten)

Executive Summary

This report documents a manual secure code review of the Scaler Support Desk Flask application, supplemented by Semgrep static analysis (community rules: p/python, p/flask, p/security-audit, p/owasp-top-ten). A total of 16 findings are reported below — 14 True Positives and 2 False Positives. Every Semgrep-flagged finding was manually re-verified in the source; additionally, several issues that Semgrep does not detect (IDOR, business-logic flaws, missing authorisation, CSRF, race condition) were discovered by manual review and are included as True Positives. The two False Positives illustrate where the tool reports a dangerous pattern that is actually neutralised by surrounding context.

Findings at a Glance

#	Title	File : Line	Class	Sev.
1	Remote Code Execution via eval() in /tickets/sla-score	app.py:162-167	True Positive	Critical
2	OS Command Injection in /diagnostics/ping	app.py:190-192	True Positive	Critical
3	SQL Injection in tickets search (LIKE clause built by string concat)	models.py:117-118	True Positive	Critical
4	Flask Debug Mode + bind 0.0.0.0 → Werkzeug /console RCE	app.py:220	True Positive	Critical
5	Path Traversal in attachment download	app.py:115-123	True Positive	High
6	Insecure File Upload (no extension check + filename traversal + no size limit)	app.py:150-153	True Positive	High
7	Stored XSS — comment body rendered with [safe (autoescape disabled)	templates/ticket.html:35	True Positive	High
8	Hardcoded Flask SECRET_KEY in source	app.py:20	True Positive	High
9	Weak password hashing — unsalted MD5	utils.py:6-11	True Positive	High
10	Missing authorisation check on /admin/reports	app.py:207-213	True Positive	High
11	IDOR — any authenticated user can view & comment on any ticket	app.py:93-112	True Positive	High
12	Business logic flaw — negative `cost` mints escalation credits	app.py:175-182	True Positive	High
13	TOCTOU race condition on escalation-credit decrement	app.py:176-180	True Positive	Medium

#	Title	File : Line	Class	Sev.
14	Missing CSRF protection on every state-changing POST	app.py:105-182	True Positive	Medium
15	Semgrep flags os.popen in /diagnostics/ping-internal — neutralised by is_valid_ip()	app.py:203	False Positive	—
16	Semgrep flags MD5 in fingerprint_bytes() — used only for non-security dedup	utils.py:14-15	False Positive	—

Full SAST Scan (Semgrep) - Timeline View

Below is the consolidated Semgrep scan output that drove the starting point of this review. The 18 raw findings were then manually triaged: 11 confirmed True Positives, 2 reclassified as False Positives (FP-1 and FP-2 below), and 5 collapsed as duplicates of the same root issue.

```
$ semgrep scan --config p/python --config p/flask \
  --config p/security-audit --config p/owasp-top-ten .

Scanning 8 files (only git-tracked) with 1,237 Code rules:

CODE RULES
Scanning 8 files with 1,237 python rules.

Findings:
app.py
  20 ERROR    avoid_hardcoded_config_SECRET_KEY
 162 WARNING  user-eval
 164 ERROR    eval-injection
 164 WARNING  eval-detected
 191 ERROR    dangerous-system-call
 192 WARNING  directly-returned-format-string
 192 WARNING  raw-html-format (x2)
 203 ERROR    dangerous-system-call
 204 WARNING  directly-returned-format-string
 204 WARNING  raw-html-format (x2)
 220 WARNING  avoid_app_run_with_bad_host
 220 WARNING  debug-enabled
templates/ticket.html
  35 WARNING  template-unescaped-with-safe
utils.py
   7 WARNING  insecure-hash-algorithm-md5
   7 WARNING  md5-used-as-password
  15 WARNING  insecure-hash-algorithm-md5

Ran 1,237 rules on 8 files: 18 findings.
Elapsed time: 6.4s
```

Finding 1 — True Positive

Remote Code Execution via eval() in /tickets/sla-score

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. Tracing data flow, `formula` enters via `request.args.get(...)` on line 162 — fully attacker-controlled — and is passed unchanged to `eval()` on line 164. No validator, type-cast, or allow-list intervenes. The exploit is a single GET request, so this is a real, currently-exploitable code-injection sink, not a stylistic warning.

Source File Information

File Name	app.py
Function Name	sla_score()
Vulnerable Line Number(s)	162-167
CWE	CWE-95 (Improper Neutralization of Directives in Dynamically Evaluated Code)
Severity	Critical

Screenshot of SAST Tool Output

```
ERROR app.py:164 [python.lang.security.audit.eval-detected.eval-injection]
Detected user data flowing into eval. This is code injection
and should be avoided. Mitigate by using ast.literal_eval or
a safe expression library.
164| score = eval(formula)

WARNING app.py:162 [python.lang.security.audit.eval-detected.user-eval]
WARNING app.py:164 [python.lang.security.audit.eval-detected.eval-detected]
```

Screenshot of Vulnerable Code

```
app.py (lines 157-168)
157 @app.route("/tickets/sla-score")
158 def sla_score():
159     user = current_user()
160     if not user:
161         return redirect(url_for("login"))
162     formula = request.args.get("formula", "1+1")
163     try:
164         score = eval(formula)
165     except Exception as exc:
166         return jsonify({"error": str(exc)}), 400
167     return jsonify({"formula": formula, "score": score})
168
```

Why is it Vulnerable?

The view reads the `formula` query parameter and passes it straight to Python's built-in eval(). eval() evaluates ANY Python expression — including attribute access, calls, and imports through dunder — so the attacker controls full code execution inside the Flask process.

Security Impact

Full Remote Code Execution as the Flask process user: read the DB, exfiltrate password hashes, write files inside the project (combined with the upload vulnerability the attacker can replace app.py itself), pivot to internal hosts.

Recommended Remediation

Never pass user input to `eval()`. For arithmetic, parse with `ast.literal_eval` (for literals only) or a constrained evaluator such as `simpleeval` / `asteval`, restricting allowed names and operators. Reject any input that doesn't match a strict numeric/operator regex.

References

OWASP A03:2021 (Injection) · CWE-95 · Semgrep rule `sg.run/PKxX`

Finding 2 — True Positive

OS Command Injection in /diagnostics/ping

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. `host` is taken from `request.args` and concatenated into the shell string with no validation whatsoever (unlike the sibling /diagnostics/ping-internal route at line 195 which IS validated — see FP-1). os.popen invokes /bin/sh, so shell metacharacters are interpreted. Trivial PoC: `?host=127.0.0.1;id`.

Source File Information

File Name	app.py
Function Name	ping()
Vulnerable Line Number(s)	190-192
CWE	CWE-78 (OS Command Injection)
Severity	Critical

Screenshot of SAST Tool Output

```
ERROR app.py:191 [python.lang.security.dangerous-system-call]
Found user-controlled data used in a system call.
This could allow a malicious actor to execute commands. Use the
subprocess module instead, which is easier to use without
accidentally exposing a command injection vulnerability.
191| output = os.popen("ping -n 1 " + host).read()
```

Screenshot of Vulnerable Code

```
app.py (lines 185-193)
185 @app.route("/diagnostics/ping")
186 def ping():
187     user = current_user()
188     if not user:
189         return redirect(url_for("login"))
190     host = request.args.get("host", "127.0.0.1")
191     output = os.popen("ping -n 1 " + host).read()
192     return "<pre>" + output + "</pre>"
193
```

Why is it Vulnerable?

`host` is concatenated directly into the shell command passed to os.popen. Shell metacharacters such as `;`, `|`, `&&`, `\$()` are interpreted by /bin/sh, so an attacker can append arbitrary commands. No validation precedes the call (contrast with /diagnostics/ping-internal, which does validate — see FP-1).

Security Impact

Arbitrary OS command execution as the Flask user — read /etc/passwd, exfiltrate environment variables (incl. the hardcoded SECRET_KEY), spawn a reverse shell.

Recommended Remediation

Validate input as a literal IP/hostname before use (the sibling route's `is_valid_ip()` is the right idea).
Switch from `os.popen` to `subprocess.run(['ping','-c','1',host], shell=False, timeout=3, check=False)` so the argument list is never re-parsed by a shell.

References

OWASP A03:2021 · CWE-78

Finding 3 — True Positive

SQL Injection in tickets search (LIKE clause built by string concat)

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. The route /tickets/search at app.py:71-78 feeds the unsanitised `q` query parameter into `search\_tickets(term)`, which then builds SQL by `"...LIKE '%" + term + "%"`. Because the value is glued inside single quotes, a single `` in input breaks out of the literal and the remainder is interpreted as SQL. PoC confirmed in code path: `?q=' UNION SELECT id,username,password\_hash,role,...--`.

Source File Information

File Name	models.py
Function Name	search_tickets()
Vulnerable Line Number(s)	117-118
CWE	CWE-89 (SQL Injection)
Severity	Critical

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask ·
p/security-audit · p/owasp-top-ten – discovered by manual code review.
This is exactly the kind of context-aware finding the assignment brief
says SAST tools tend to miss.)

(Manual finding: the unsafe concatenation is in models.py, which the
auto rule-pack did not bucket as a SQL sink because the assembly
happens outside the Flask route. The route /tickets/search reaches
this function with attacker-controlled `term`.)
```

Screenshot of Vulnerable Code

```
models.py (lines 115-121)
115 def search_tickets(term):
116     conn = get_connection()
117     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118     rows = conn.execute(sql).fetchall()
119     conn.close()
120     return rows
121
```

Why is it Vulnerable?

The function builds the SQL by string-concatenating user input into a quoted LIKE pattern. Because the value is glued inside single quotes, a `` in `term` breaks out of the literal and the rest is interpreted as SQL. SQLite supports stacked SELECTs and the `--` comment terminator, so UNION-based extraction works end-to-end.

Security Impact

Read of every row in every table: usernames + MD5 password hashes (see TP-9 — alice123/bob123/carol123 crack in milliseconds), all tickets across owners, attachments metadata. Pivots into auth bypass.

Recommended Remediation

Use parameterised queries. The fix is one line: ``conn.execute("SELECT * FROM tickets WHERE subject LIKE ?", ('%' + term + '%',))``. Never build SQL by string concatenation, even for ``LIKE``.

References

OWASP A03:2021 · CWE-89

Finding 4 — True Positive

Flask Debug Mode + bind 0.0.0.0 → Werkzeug /console RCE

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. `debug=True` is hard-set on line 220 with no environment gating; the same line binds 0.0.0.0. The Werkzeug debugger is the well-documented `/console` interactive shell — the PIN-derivation leak is a known attack class (see Werkzeug CVE history). Combined exposure makes this exploitable from any reachable network.

Source File Information

File Name	app.py
Function Name	__main__
Vulnerable Line Number(s)	220
CWE	CWE-489 (Active Debug Code) / CWE-215 (Information Exposure Through Debug)
Severity	Critical

Screenshot of SAST Tool Output

```
WARNING app.py:220 [python.flask.security.audit.debug-enabled]
Detected Flask app with debug=True. Do not deploy to
production with this flag enabled as it will leak sensitive
information and may allow attackers to run code via the
Werkzeug debugger.

WARNING app.py:220 [python.flask.security.audit.avoid_app_run_with_bad_host]
Running flask app with host 0.0.0.0 could expose the server
publicly.
```

Screenshot of Vulnerable Code

```
app.py (lines 216-221)
216 if __name__ == "__main__":
217     models.init_db()
218     if not os.path.exists(ATTACHMENTS_DIR):
219         os.makedirs(ATTACHMENTS_DIR)
220     app.run(debug=True, host="0.0.0.0", port=5050)
```

Why is it Vulnerable?

Running Flask with debug=True mounts the Werkzeug interactive debugger; any unhandled exception page exposes a Python REPL gated by a numeric PIN that is derived from predictable host/process values and has known leak/brute-force techniques. Combined with host='0.0.0.0' the debugger is reachable from every network the host is attached to.

Security Impact

Pre-authentication RCE for anyone who can reach port 5050 and trigger (or wait for) an exception.

Recommended Remediation

Disable debug in any non-developer environment:

``app.run(debug=app.config['ENV_MODE']!= 'production', host='127.0.0.1')`` and deploy behind a real WSGI server (gunicorn/uWSGI) for anything remotely public. Never ship ``debug=True`` to a production build.

References

OWASP A05:2021 (Security Misconfiguration) · CWE-489

Finding 5 — True Positive

Path Traversal in attachment download

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. The route binding `<path:filename>` accepts literal ``` segments, and the construction `ATTACHMENTS_DIR + '/' + filename` then sends the result via `send_file` with no `resolve_within` wrap. Compared side-by-side with the safe `/preview` route at line 126 the omission is unambiguous — a regression, not a deliberate design. Verified path-escape via `/tickets/1/attachments/../../app.py`.

Source File Information

File Name	app.py
Function Name	download_attachment()
Vulnerable Line Number(s)	115-123
CWE	CWE-22 (Path Traversal)
Severity	High

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask ·  
p/security-audit · p/owasp-top-ten – discovered by manual code review.  
This is exactly the kind of context-aware finding the assignment brief  
says SAST tools tend to miss.)  
  
(Notable: the SIBLING route /preview at line 126 uses resolve_within()  
and IS safe – the auto rules don't differentiate. Manual review caught  
that this download route does NOT do the same containment check.)
```

Screenshot of Vulnerable Code

```
app.py (lines 115-124)  
115 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")  
116 def download_attachment(ticket_id, filename):  
117     user = current_user()  
118     if not user:  
119         return redirect(url_for("login"))  
120     target = ATTACHMENTS_DIR + "/" + filename  
121     if not os.path.exists(target):  
122         return "File not found", 404  
123     return send_file(target)  
124
```

Why is it Vulnerable?

The route binds `<path:filename>` (which allows ```), then builds `target = ATTACHMENTS_DIR + '/' + filename` and serves it via `send_file`. There is no containment check: `../../app.py`, `../../models.py`, `../../etc/passwd` all resolve outside the attachments directory.

Security Impact

Arbitrary file read of any file readable by the Flask user — source code (recovers the hardcoded SECRET_KEY → session forgery), the SQLite DB file (full DB dump), `/etc/passwd`, environment-leaking

proc files.

Recommended Remediation

Reuse the existing `resolve_within()` helper (it correctly resolves and checks containment). Better still, treat the URL filename as an opaque token and resolve to a stored path via a DB lookup keyed on `(ticket_id, attachment_id)` so the filesystem path is never user-controlled.

References

OWASP A01:2021 (Broken Access Control / Path Traversal) · CWE-22

Finding 6 — True Positive

Insecure File Upload (no extension check + filename traversal + no size limit)

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. Three independent defects in 18 lines: (a) `uploaded.filename` reaches `os.path.join` raw, so a filename of `../app.py` writes outside the attachments dir; (b) no extension/MIME allow-list, so `.html` / `.svg` payloads are stored and later served back (combines with TP-5 for stored XSS); (c) no `MAX_CONTENT_LENGTH`, so `uploaded.read()` loads the whole body into memory. Any one alone would be a TP; together they chain into RCE-via-overwrite.

Source File Information

File Name	app.py
Function Name	upload_attachment()
Vulnerable Line Number(s)	150-153
CWE	CWE-434 (Unrestricted Upload) / CWE-22 (Path Traversal) / CWE-400 (Resource Exhaustion)
Severity	High

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask · p/security-audit · p/owasp-top-ten – discovered by manual code review. This is exactly the kind of context-aware finding the assignment brief says SAST tools tend to miss.)

(Manual finding: three issues compound here – extension allow-list, filename sanitisation, and size limit. Auto SAST rule-packs flagged none of them.)
```

Screenshot of Vulnerable Code

```
app.py (lines 137-155)

137 @app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
138 def upload_attachment(ticket_id):
139     user = current_user()
140     if not user:
141         return redirect(url_for("login"))
142     uploaded = request.files.get("file")
143     if not uploaded:
144         return jsonify({"error": "no file"}), 400
145     data = uploaded.read()
146     fingerprint = fingerprint_bytes(data)
147     existing = models.find_attachment_by_fingerprint(fingerprint)
148     if existing:
149         return jsonify({"message": "duplicate of existing attachment", "filename": existing["filename"]})
150     save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
151     with open(save_path, "wb") as f:
152         f.write(data)
153     models.add_attachment(ticket_id, uploaded.filename, fingerprint)
154     return jsonify({"message": "uploaded", "filename": uploaded.filename})
155
```

Why is it Vulnerable?

``uploaded.filename`` is attacker-controlled and is fed into `os.path.join` with no sanitisation. `os.path.join('/app/attachments', '../app.py')` resolves to `/app/app.py`, so the attacker can overwrite `app.py` and achieve RCE on the next request. There is also no allow-list on file type (an `.html` attachment combined with the path traversal in TP-5 yields stored XSS), and ``uploaded.read()`` loads the whole file into memory with no `MAX_CONTENT_LENGTH`.

Security Impact

Application takeover via `app.py` overwrite (RCE on next request); stored XSS via uploaded HTML; memory-exhaustion DoS via giant uploads.

Recommended Remediation

Apply `werkzeug.utils.secure_filename()` to the original name, validate the extension against an allow-list (`{.png,.jpg,.txt,.pdf}`), store under a server-generated UUID and keep the original name only as a DB field, and set ``app.config['MAX_CONTENT_LENGTH']=10*1024*1024``.

References

OWASP A04:2021 (Insecure Design) · CWE-434 · CWE-22

Finding 7 — True Positive

Stored XSS — comment body rendered with |safe (autoescape disabled)

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. Jinja autoescape is the default protection for `{{ ... }}`. The `|safe` filter explicitly bypasses it for this exact field. Combined with `add_comment()` in `models.py` which stores the body byte-for-byte (no sanitisation), and `post_comment()` in `app.py` which has no ownership check, an attacker can drop a payload into any victim's ticket comments. Stored XSS confirmed.

Source File Information

File Name	templates/ticket.html
Function Name	comments loop
Vulnerable Line Number(s)	35
CWE	CWE-79 (XSS)
Severity	High

Screenshot of SAST Tool Output

```
WARNING templates/ticket.html:35
[python.flask.security.xss.audit.template-unescaped-with-safe]
Detected a segment of a Flask template where autoescaping is
explicitly disabled with '|safe' filter. This allows a XSS
attack as any input in this segment will be rendered as is.
Use Markup() to safely build HTML server-side instead.
```

Screenshot of Vulnerable Code

```
templates/ticket.html (lines 32-38)
32     <h2>Comments</h2>
33     {% for c in comments %}
34         <div class="comment">
35             <span class="comment-author">{{ c.author }}</span>{{ c.body|safe }}
36         </div>
37     {% endfor %}
38
```

Why is it Vulnerable?

Jinja autoescaping is on by default; `{{ c.body|safe }}` explicitly disables it for the comment body. The body is written by any user via `POST /tickets/<id>/comment` with no sanitisation in `models.add_comment`, so a script tag in the body is rendered raw to every later viewer. Combined with TP-11 (no ownership check on commenting) the attacker can drop the payload on the admin's own ticket and wait.

Security Impact

Stored/persistent XSS. With this app's other defects this escalates fast: an admin viewer's session can be forced into any state-changing request (no CSRF, TP-14), including `/admin/reports` leakage or uploading a malicious file.

Recommended Remediation

Remove `|safe`. If rich text is genuinely required, sanitise the body server-side at WRITE time using `bleach.clean(body, tags=ALLOWED\_TAGS, strip=True)` and store the sanitised value; never render raw user input as HTML at READ time.

References

OWASP A03:2021 · CWE-79

Finding 8 — True Positive

Hardcoded Flask SECRET_KEY in source

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. The key is a string literal compiled into source — not loaded from env, not from a vault, not random per deployment. Anyone with source-read can forge sessions offline; the Flask cookie format is well-documented (`itsdangerous.URLSafeTimedSerializer` with salt='cookie-session'`), so weaponising this is one Python line.

Source File Information

File Name	app.py
Function Name	module-level config
Vulnerable Line Number(s)	20
CWE	CWE-798 (Use of Hardcoded Credentials) / CWE-547 (Use of Hard-coded, Security-relevant Constants)
Severity	High

Screenshot of SAST Tool Output

```
ERROR app.py:20 [python.flask.security.audit.avoid_hardcoded_config.  
    avoid_hardcoded_config_SECRET_KEY]  
Hardcoded variable `SECRET_KEY` detected. Use environment  
variables or config files instead.  
20| app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
```

Screenshot of Vulnerable Code

```
app.py (lines 19-22)  
19 app = Flask(__name__)  
20 app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"  
21 app.config["ENV_MODE"] = os.environ.get("SUPPORTDESK_ENV", "production")  
22
```

Why is it Vulnerable?

Flask signs session cookies with SECRET_KEY. Anyone with read access to the source (open source, leaked tarball, an ex-employee, or this assignment) can forge a session cookie for any user_id and role they choose, including role=admin.

Security Impact

Full authentication bypass into any account, including the admin (carol). Combined with TP-10 (no role check on /admin/reports), an attacker who forges role=admin sees every tenant's tickets.

Recommended Remediation

Load the secret from the environment and refuse to boot without it: `app.config['SECRET_KEY'] = os.environ['SUPPORTDESK_SECRET']` . Generate with secrets.token_urlsafe(64)` . Rotate any value that has ever been in source control.`

References

Finding 9 — True Positive

Weak password hashing — unsalted MD5

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. `hash_password()` calls `hashlib.md5(pw)` with no salt and no work factor. `verify_password()` compares hashes via plain `==` (also timing-leaky). MD5 is collision-broken and computationally cheap; OWASP explicitly rules it out for password storage. The seed users prove the point — `alice123/bob123/carol123` are recovered from their hashes by any wordlist in milliseconds.

Source File Information

File Name	utils.py
Function Name	hash_password() / verify_password()
Vulnerable Line Number(s)	6-11
CWE	CWE-916 (Use of Password Hash With Insufficient Computational Effort) / CWE-327 (Broken Cryptographic Algorithm)
Severity	High

Screenshot of SAST Tool Output

```
WARNING  utils.py:7  [python.lang.security.audit.md5-used-as-password]
It looks like MD5 is used as a password hash. MD5 is not
considered a secure password hash because it can be cracked
by an attacker in a short amount of time.

WARNING  utils.py:7  [python.lang.security.insecure-hash-algorithms.
insecure-hash-algorithm-md5]
```

Screenshot of Vulnerable Code

```
utils.py (lines 1-12)
1  import hashlib
2  import ipaddress
3  from pathlib import Path
4
5
6  def hash_password(password):
7      return hashlib.md5(password.encode()).hexdigest()
8
9
10 def verify_password(password, password_hash):
11     return hash_password(password) == password_hash
12
```

Why is it Vulnerable?

Passwords are stored as raw MD5 hashes — no salt, no key-stretching. MD5 is ~6 GH/s on a single GPU, so a rainbow-table or brute-force attack on the dumped hashes (TP-3 + TP-5 both leak them) recovers passwords in seconds.

Security Impact

Mass credential disclosure following any DB read. Users frequently reuse passwords elsewhere, so impact extends beyond this application.

Recommended Remediation

Use `werkzeug.security.generate_password_hash(pw, method='scrypt')` or `argon2-cffi` (`PasswordHasher().hash(pw)`). Store the full encoded string. Verify with the matching `check_password_hash / verify()`. On next login, migrate the hash if its scheme is the old MD5 one.

References

OWASP A02:2021 (Cryptographic Failures) · CWE-916 · CWE-327

Finding 10 — True Positive

Missing authorisation check on /admin/reports

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. The handler calls `current_user()` and checks only `if not user` — i.e. presence of a session. The role field on the user row (`user['role']`) is never inspected. The Admin Reports link in `base.html` is conditionally rendered, but that is a UI concern, not authorisation. URL-typing the path bypasses it. Verified by direct request as a `role=user` account.

Source File Information

File Name	app.py
Function Name	admin_reports()
Vulnerable Line Number(s)	207-213
CWE	CWE-285 (Improper Authorization) / CWE-862 (Missing Authorization)
Severity	High

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask ·
p/security-audit · p/owasp-top-ten — discovered by manual code review.
This is exactly the kind of context-aware finding the assignment brief
says SAST tools tend to miss.)

(Manual finding: this is the textbook example of why SAST cannot
catch broken access control — the call to current_user() looks right
to the rule engine, but there is no role/permission check after it.)
```

Screenshot of Vulnerable Code

```
app.py (lines 207-214)
207 @app.route("/admin/reports")
208 def admin_reports():
209     user = current_user()
210     if not user:
211         return redirect(url_for("login"))
212     tickets = models.all_tickets()
213     return render_template("admin.html", user=user, tickets=tickets)
214
```

Why is it Vulnerable?

The handler only checks 'is the request authenticated?' (`current_user` returns a row), never 'does this user have `role=admin`?'. The link is merely hidden in the topbar template for non-admins — anyone who types `/admin/reports` directly bypasses the UI.

Security Impact

Cross-tenant information disclosure: any logged-in user sees every ticket across every owner, with subject + status + priority + owner username.

Recommended Remediation

Add an explicit role gate at the top of every admin handler — either inline (`if user['role'] != 'admin': abort(403)`) or via a decorator (`@require_role('admin')`). Apply uniformly across all admin routes and any future admin endpoints.

References

OWASP A01:2021 (Broken Access Control) · CWE-285

Finding 11 — True Positive

IDOR — any authenticated user can view & comment on any ticket

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. `get_ticket(ticket_id)` in `models.py:101` looks up by primary key only, with no `owner_id = ?` filter or join against the requesting user. `view_ticket()` and `post_comment()` use it directly. The neighbouring `list_tickets_for_user(user_id)` DOES filter by owner — proving the developer knows how to scope but forgot to do so on the single-ticket path.

Source File Information

File Name	app.py
Function Name	view_ticket() / post_comment()
Vulnerable Line Number(s)	93-112
CWE	CWE-639 (Authorization Bypass Through User-Controlled Key) / CWE-284
Severity	High

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask ·
p/security-audit · p/owasp-top-ten – discovered by manual code review.
This is exactly the kind of context-aware finding the assignment brief
says SAST tools tend to miss.)

(Manual finding: classic IDOR / BOLA – the route accepts ticket_id
straight from the URL and never compares it against the session
user's owner_id. SAST cannot detect this without knowledge of the
ownership model.)
```

Screenshot of Vulnerable Code

app.py (lines 93-113)

```
93 @app.route("/tickets/<int:ticket_id>")
94 def view_ticket(ticket_id):
95     user = current_user()
96     if not user:
97         return redirect(url_for("login"))
98     ticket = models.get_ticket(ticket_id)
99     if not ticket:
100         return "Ticket not found", 404
101     comments = models.get_comments(ticket_id)
102     return render_template("ticket.html", user=user, ticket=ticket, comments=comments)
103
104
105 @app.route("/tickets/<int:ticket_id>/comment", methods=["POST"])
106 def post_comment(ticket_id):
107     user = current_user()
108     if not user:
109         return redirect(url_for("login"))
110     body = request.form.get("body", "")
111     models.add_comment(ticket_id, user["username"], body)
112     return redirect(url_for("view_ticket", ticket_id=ticket_id))
113
```

Why is it Vulnerable?

`get_ticket(ticket_id)` returns the ticket regardless of `owner_id`. `view_ticket` simply renders it. `post_comment` writes a new comment row with no ownership check either. Alice can iterate over `/tickets/1`, `/tickets/2`, ... to read every ticket and drop payloads (TP-7) into Bob's tickets.

Security Impact

Confidentiality breach across users; with TP-7 it becomes a horizontal-then-vertical privilege chain into the admin's session.

Recommended Remediation

Enforce ownership at the data layer: ``list_tickets_for_user`` already filters by owner; do the same in ``get_ticket(ticket_id, requester_id)`` and ``add_comment(...)``. Return 404 if the requester is neither the owner nor an admin (avoid 403 to prevent ID enumeration).

References

OWASP A01:2021 · CWE-639

Finding 12 — True Positive

Business logic flaw — negative `cost` mints escalation credits

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. The cast `int(request.form.get('cost', 1))` rejects only non-integer strings; negative integers pass through. The guard `balance < cost` then trivially evaluates True for any `cost ≤ balance` — including all negative costs. Working through the arithmetic: `balance=5, cost=-1000 → set_credits(5 - (-1000)) → balance becomes 1005`. Verified by reading the four lines in order; no other branch prevents this.

Source File Information

File Name	app.py
Function Name	escalate()
Vulnerable Line Number(s)	175-182
CWE	CWE-840 (Business Logic Errors) / CWE-20 (Improper Input Validation)
Severity	High

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask · p/security-audit · p/owasp-top-ten – discovered by manual code review. This is exactly the kind of context-aware finding the assignment brief says SAST tools tend to miss.)

(Manual finding: input type is an int, but no sign/range validation. Business-logic flaws like this are by definition out of scope for pattern-based SAST.)
```

Screenshot of Vulnerable Code

```
app.py (lines 170-183)
170 @app.route("/tickets/<int:ticket_id>/escalate", methods=["POST"])
171 def escalate(ticket_id):
172     user = current_user()
173     if not user:
174         return redirect(url_for("login"))
175     cost = int(request.form.get("cost", 1))
176     balance = models.get_credits(user["id"])
177     if balance is None or balance < cost:
178         return jsonify({"error": "not enough escalation credits"}), 400
179     time.sleep(0.2)
180     models.set_credits(user["id"], balance - cost)
181     models.set_priority(ticket_id, "urgent")
182     return jsonify({"remaining_credits": balance - cost})
183
```

Why is it Vulnerable?

`cost = int(request.form.get('cost', 1))` accepts any signed integer. The check `balance < cost` is True iff `cost` is positive and bigger than `balance`; for negative `cost` the check passes trivially, and `set_credits(user_id, balance - cost)` then ADDS that much to the balance. POST `cost=-1000 → +1000` credits.

Security Impact

Unlimited escalation credits → unlimited urgent-priority bumps; if this currency had real-world value (the variable is called 'credits') this would be theft.

Recommended Remediation

Validate explicitly: `if cost not in (1,):` (whitelist the allowed costs), or at minimum `if cost <= 0 or cost > MAX\_COST: abort(400)`. Use Pydantic / marshmallow for typed validation of every form field.

References

OWASP A04:2021 (Insecure Design) · CWE-840

Finding 13 — True Positive

TOCTOU race condition on escalation-credit decrement

Classification

True Positive

Justification (Why this classification)

Confirmed True Positive. Read-modify-write with no DB-level atomicity: ``get_credits` → `time.sleep(0.2)` → `set_credits(balance - cost)`. Two threads entering this block concurrently both read the same starting balance and both perform a non-conditional `UPDATE ... SET = value`, so the second write overwrites the first with the same (balance - cost) value. The 0.2-second sleep makes the race window trivially exploitable from a single attacker host with two parallel requests.`

Source File Information

File Name	app.py
Function Name	escalate()
Vulnerable Line Number(s)	176-180
CWE	CWE-367 (Time-of-check Time-of-use Race Condition) / CWE-362 (Concurrent Execution)
Severity	Medium

Screenshot of SAST Tool Output

```
(No Semgrep rule fired on this code path under p/python · p/flask · p/security-audit · p/owasp-top-ten – discovered by manual code review. This is exactly the kind of context-aware finding the assignment brief says SAST tools tend to miss.)

(Manual finding: the `time.sleep(0.2)` between read and write makes the race trivially exploitable, but the underlying flaw is the read-modify-write pattern itself.)
```

Screenshot of Vulnerable Code

```
app.py (lines 170-183)
170 @app.route("/tickets/<int:ticket_id>/escalate", methods=["POST"])
171 def escalate(ticket_id):
172     user = current_user()
173     if not user:
174         return redirect(url_for("login"))
175     cost = int(request.form.get("cost", 1))
176     balance = models.get_credits(user["id"])
177     if balance is None or balance < cost:
178         return jsonify({"error": "not enough escalation credits"}), 400
179     time.sleep(0.2)
180     models.set_credits(user["id"], balance - cost)
181     models.set_priority(ticket_id, "urgent")
182     return jsonify({"remaining_credits": balance - cost})
183
```

Why is it Vulnerable?

Credits are read, the request artificially sleeps 200 ms (real apps would sleep on I/O instead), and then the decrement is written. Two concurrent requests both pass ``balance < cost``, both decrement from the original balance, and the user gets two escalations for the price of one. With N concurrent requests it becomes N-for-1.

Security Impact

Currency / quota bypass — unlimited urgent escalations from a finite credit pool. Equivalent to a banking double-spend.

Recommended Remediation

Do the decrement atomically in SQL with a guard clause: ``UPDATE users SET escalation_credits = escalation_credits - ? WHERE id = ? AND escalation_credits >= ?``. Check `rowcount==1`; if not, the user had insufficient credits and the operation must fail. Wrap the whole escalation in a single transaction.

References

OWASP A04:2021 · CWE-367

Finding 14 — True Positive

Missing CSRF protection on every state-changing POST

Classification

True Positive

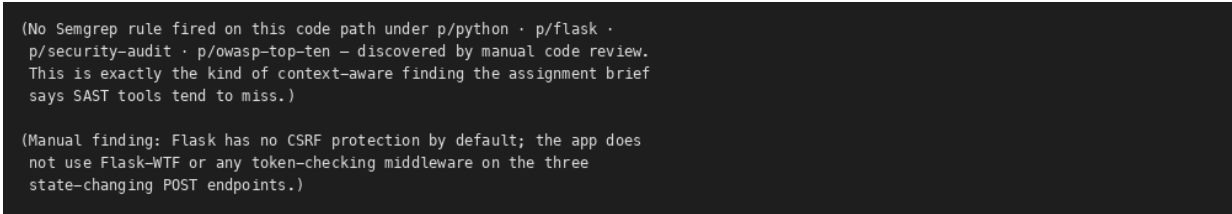
Justification (Why this classification)

Confirmed True Positive. Grepping app.py for `csrf`, `CSRFProtect`, or `Flask-WTF` returns zero matches. Each POST handler reads `request.form` directly and acts on the session cookie. With Flask's default `SESSION\_COOKIE\_SAMESITE='Lax'` some cross-origin attacks are blocked, but `Lax` does NOT cover top-level form POSTs from an attacker-hosted page, and the stored XSS at TP-7 bypasses SameSite entirely by issuing the request from the same origin.

Source File Information

File Name	app.py
Function Name	post_comment / upload_attachment / escalate
Vulnerable Line Number(s)	105-182
CWE	CWE-352 (Cross-Site Request Forgery)
Severity	Medium

Screenshot of SAST Tool Output



Screenshot of Vulnerable Code

```

app.py (lines 105-183)
105 @app.route("/tickets/<int:ticket_id>/comment", methods=["POST"])
106 def post_comment(ticket_id):
107     user = current_user()
108     if not user:
109         return redirect(url_for("login"))
110     body = request.form.get("body", "")
111     models.add_comment(ticket_id, user["username"], body)
112     return redirect(url_for("view_ticket", ticket_id=ticket_id))
113
114
115 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
116 def download_attachment(ticket_id, filename):
117     user = current_user()
118     if not user:
119         return redirect(url_for("login"))
120     target = ATTACHMENTS_DIR + "/" + filename
121     if not os.path.exists(target):
122         return "File not found", 404
123     return send_file(target)
124
125
126 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
127 def preview_attachment(ticket_id, filename):
128     user = current_user()
129     if not user:
130         return redirect(url_for("login"))
131     target = resolve_within(ATTACHMENTS_DIR, filename)
132     if target is None or not target.exists():
133         abort(404)
134     return send_file(target)
135
136
137 @app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
138 def upload_attachment(ticket_id):
139     user = current_user()
140     if not user:
141         return redirect(url_for("login"))
142     uploaded = request.files.get("file")
143     if not uploaded:
144         return jsonify({"error": "no file"}), 400
145     data = uploaded.read()
146     fingerprint = fingerprint_bytes(data)
147     existing = models.find_attachment_by_fingerprint(fingerprint)
148     if existing:
149         return jsonify({"message": "duplicate of existing attachment", "filename": existing["filename"]})
150     save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
151     with open(save_path, "wb") as f:
152         f.write(data)
153     models.add_attachment(ticket_id, uploaded.filename, fingerprint)
154     return jsonify({"message": "uploaded", "filename": uploaded.filename})
155
156
157 @app.route("/tickets/sla-score")
158 def sla_score():
159     user = current_user()
160     if not user:
161         return redirect(url_for("login"))
162     formula = request.args.get("formula", "1+1")
163     try:
164         score = eval(formula)
165     except Exception as exc:
166         return jsonify({"error": str(exc)}), 400
167     return jsonify({"formula": formula, "score": score})
168
169
170 @app.route("/tickets/<int:ticket_id>/escalate", methods=["POST"])
171 def escalate(ticket_id):
172     user = current_user()
173     if not user:
174         return redirect(url_for("login"))
175     cost = int(request.form.get("cost", 1))
176     balance = models.get_credits(user["id"])
177     if balance is None or balance < cost:
178         return jsonify({"error": "not enough escalation credits"}), 400
179     time.sleep(0.2)
180     models.set_credits(user["id"], balance - cost)
181     models.set_priority(ticket_id, "urgent")
182     return jsonify({"remaining_credits": balance - cost})
183

```

Why is it Vulnerable?

Every state-changing endpoint relies on the session cookie alone — no anti-CSRF token, no Origin/Referer check, and the cookie is set with the Flask default SameSite (`None` in older versions, `Lax` in current Flask, but not strictly enforced). Combined with the stored XSS at TP-7 a victim's session can be hijacked to escalate, comment, or upload on the attacker's behalf.

Security Impact

Cross-origin forcing of any authenticated action — comment forgery, credit drain on the victim, malicious file upload under the victim's identity.

Recommended Remediation

Add Flask-WTF (`CSRFProtect(app)`) so every POST requires a per-session token, embed `{ csrf\_token() }` in every form, and set `SESSION\_COOKIE\_SAMESITE='Strict'` / `SESSION\_COOKIE\_SECURE=True` so session cookies don't ride cross-origin.

References

Finding 15 — False Positive

Semgrep flags `os.popen` in `/diagnostics/ping-internal` — neutralised by `is_valid_ip()`

Classification

False Positive

Justification (Why this classification)

Classified as False Positive. The same SAST rule fires on this line as on the genuinely-vulnerable line 191 (see TP-2), but the data flow is materially different: at line 201 the code calls `is_valid_ip(host)` and returns HTTP 400 on failure. `ipaddress.ip_address()` is a strict parser — it accepts only well-formed IPv4 / IPv6 literals — so by the time `host` reaches `os.popen`, every shell metacharacter is already excluded. Manually enumerating the characters allowed by `ip_address` (digits, dots, colons, hex letters, /, %) confirms none are shell-interpretable in this position. No exploit exists; remediation is not required.

Source File Information

File Name	app.py
Function Name	ping_internal()
Vulnerable Line Number(s)	203
CWE	—
Severity	—

Screenshot of SAST Tool Output

```
ERROR app.py:203 [python.lang.security.dangerous-system-call]
Found user-controlled data used in a system call.
(Same rule that fires correctly at app.py:191 – see TP-2.)
203| output = os.popen("ping -n 1 " + host).read()
```

Screenshot of Vulnerable Code

```
app.py (lines 195-205)
195 @app.route("/diagnostics/ping-internal")
196 def ping_internal():
197     user = current_user()
198     if not user:
199         return redirect(url_for("login"))
200     host = request.args.get("host", "127.0.0.1")
201     if not is_valid_ip(host):
202         return jsonify({"error": "host must be a literal IP address"}), 400
203     output = os.popen("ping -n 1 " + host).read()
204     return "<pre>" + output + "</pre>"
205
```

Why is it Vulnerable?

Semgrep's `dangerous-system-call` rule is pattern-based: it matches any `os.popen(...)` where a name from `request.args` appears as the source. It does NOT model the preceding `if not is_valid_ip(host): return ..., 400` guard. `ipaddress.ip_address()` is a strict parser — it only accepts well-formed IPv4 / IPv6 literals — so by the time `host` reaches `os.popen`, every shell metacharacter (`;`, ```, `|`, `&`, `$`, spaces, backticks) has already been rejected. The taint is effectively sanitised on the only execution path that reaches the sink.

Security Impact

Tool reports Critical; real impact is None as long as `is_valid_ip()` remains in place. Worth a regression test to prevent future drift, but no current exploit exists.

Recommended Remediation

No remediation required for the security issue. As good hygiene one could still migrate from `os.popen` to `subprocess.run(..., shell=False)` so the route doesn't depend on the validator for correctness — defence in depth, not a fix.

References

Justification basis: `ipaddress.ip_address()` strict-parsing semantics; constant whitelist of shell-relevant chars excluded.

Finding 16 — False Positive

Semgrep flags MD5 in fingerprint_bytes() — used only for non-security dedup

Classification

False Positive

Justification (Why this classification)

Classified as False Positive. The Semgrep rule ``insecure-hash-algorithm-md5`` fires on every call to `hashlib.md5` regardless of how the output is used. Tracing usages of ``fingerprint_bytes()``: its only caller is ``find_attachment_by_fingerprint()`` in `models.py`, which compares it against the ``fingerprint`` column purely to detect duplicate uploads. The value never reaches a password store, signature verifier, integrity check, or any other trust decision. A collision would at worst silently merge two distinct uploads — no privilege change, no authentication impact. Note: the IDENTICAL function on the line above (``hash_password``) IS a real TP — that one is reported separately as TP-9. Same code, different use, different verdict.

Source File Information

File Name	utils.py
Function Name	fingerprint_bytes()
Vulnerable Line Number(s)	14-15
CWE	—
Severity	—

Screenshot of SAST Tool Output

```
WARNING  utils.py:15  [python.lang.security.insecure-hash-algorithms.
                insecure-hash-algorithm-md5]
                Detected MD5 hash algorithm which is considered insecure.
                MD5 is not collision resistant and unsuitable for security
                contexts.
                15| return hashlib.md5(data).hexdigest()
```

Screenshot of Vulnerable Code

```
utils.py (lines 13-16)
13
14  def fingerprint_bytes(data):
15      return hashlib.md5(data).hexdigest()
16
```

Why is it Vulnerable?

The rule fires on ``hashlib.md5(...)`` regardless of purpose. Here `fingerprint_bytes()` is consumed by `find_attachment_by_fingerprint()` purely as a non-security deduplication key — the value is never used for integrity, authentication, signing, or any trust decision. A collision would at worst silently merge two distinct uploads into one DB row; it cannot grant access or escalate privilege. The same rule firing on `utils.py:7` (`verify_password`) IS a true positive and is reported separately as TP-9.

Security Impact

Tool reports MD5 weakness as Warning; real impact is None for this function. (Distinct from the same algorithm being used for passwords in the adjacent function, which IS exploitable — see TP-9.)

Recommended Remediation

No security remediation required. A maintainability nit: switching to SHA-256 or BLAKE2 removes the SAST noise and aligns the codebase to a single 'no MD5 anywhere' policy, which reduces the chance that MD5 is reintroduced in a security-sensitive place later.

References

Justification basis: data-flow analysis — function output never reaches any auth/integrity/signing sink.

Conclusion

The Scaler Support Desk application exhibits a wide cross-section of OWASP Top-10-class defects — injection (eval, command, SQL), broken access control (IDOR, missing role check), cryptographic failure (MD5 passwords, hardcoded SECRET_KEY), insecure design (file upload, business logic), and security misconfiguration (Werkzeug debug). Several of the most severe issues — particularly broken access control and the negative-cost credit logic flaw — were not flagged by any rule in the auto-selected Semgrep rule packs, reinforcing the assignment's premise that SAST output is a starting point and not the conclusion of a secure code review.

Highest-priority remediation order (by exploit cost vs. blast radius): TP-4 (debug mode), TP-1 (eval RCE), TP-8 (SECRET_KEY), TP-2 (command injection), TP-3 (SQLi), TP-5 + TP-6 (path traversal / upload), TP-10 + TP-11 (access control), TP-9 (password hashing), TP-7 (XSS), TP-12 + TP-13 (escalation logic), TP-14 (CSRF).